

AI-Assisted Offline OS Development Pipeline Architecture

1. Overview

This document describes a hybrid system where an AI-driven online development factory continuously builds and improves an operating system that is later deployed and used fully offline.

2. High-Level Architecture

The system is divided into two major parts: (a) Online AI OS Factory and (b) Offline Executable OS. The online factory uses AI agents to design, generate, test, and compile OS components. The output is a stable bootable image.

3. AI Agent System

Planner Agent: defines features and roadmap. Builder Agent: generates OS code. Reviewer Agent: checks bugs and logic issues. Tester Agent: runs QEMU-based simulations and validates builds.

4. Development Pipeline

Cycle: Plan → Generate Code → Compile → Test in Emulator → Validate → Store Stable Version → Deploy Build. This loop ensures continuous improvement and stability control.

5. System Components

Kernel development (C/Rust), Bootloader (Assembly), Emulator testing (QEMU), CI/CD automation (GitHub Actions), AI API integration (Gemini Flash).

6. Risk Management

Major risks include unstable AI-generated low-level code, infinite bad build loops, and hardware compatibility issues. Mitigation includes version control, rollback system, and strict automated testing.

7. Roadmap

Phase 1: Basic bootloader and kernel print output. Phase 2: Input handling and memory basics. Phase 3: AI-assisted module generation. Phase 4: Stable offline OS build system.

8. Minimum Viable Product (MVP)

A simple kernel that boots via QEMU, prints system status, supports basic input, and is generated through the AI pipeline with automated build testing.